

SEMÁNTICA

GRAMATICA

- Sintaxis
- **Semántica**
 - Semántica estática
 - Semántica dinámica
- **Procesamiento de los programas**
 - Intérpretes
 - Compiladores

SEMÁNTICA

- La ***semántica*** describe el **significado** de los **símbolos, palabras y frases** de un **lenguaje** ya sea lenguaje natural o lenguaje informático
- La ***semántica*** describe el **significado** de programas **sintácticamente válidos/correctos** para un lenguaje determinado
- Así se da significado a las construcciones del lenguaje

SEMÁNTICA

○ Ejemplos en C:

- `int vector [10];` (declaración)
- `if (a<b) max=a; else max=b;` (estructura de control)
- ✓ La semántica no es igual en todos los lenguajes.
- ✓ Código similar en distintos lenguajes puede dar distintas soluciones o reportar errores.
- ✓ Una cosa es formato escrito y otra que significa y como se comporta

Veamos ejemplos

SEMÁNTICA

• Ejemplo en C:

```
1  #include <stdio.h>
2  int x=9;
3  int Prueba()
4  {
5      int y;
6      y=x-1;
7      printf("%d\n",y);
8      return 0;
9  }
10 int Prueba1()
11 {
12     x=x+1;
13     printf("%d\n",x);
14     return 0;
15 }
16
17 int main()
18 {
19     Prueba();
20     Prueba1();
21     printf("Despues de la llamada a ambas funciones\n%d\n",x);
22     return 0;
23 }
```

```
sh-4.3$ main
```

```
8
```

```
10
```

```
Despues de llamar a prueba
```

```
10
```

```
sh-4.3$
```

El significado en C

- Es Obligatorio declarar variables y tipos.
- En este caso el **alcance** de la variable es **Global**. (El **alcance** de una variable es la parte del programa donde la variable es accesible)
- Se ve que tanto **Prueba** como **Prueba1** usan la variable **x**. En un caso para **tomar** su valor. En el otro para **modificar** su valor.

Veamos que pasa en otro lenguaje

SEMÁNTICA

• Ejemplo en Python similar

```
Archivo  Editar  Buscar  Ver  Documento  Proyecto  Construir  Herrami
Nuevo  Abrir  Guardar  Guardar todo  Revertir  Cerrar

Símbolos  Documentos  pruebaSemantica.py x
Funciones
  pruebaSemantica [
  pruebaSemantica1
Variables
  x [8]

1 def pruebaSemantica():
2     y = x - 1
3     print ( y )
4 def pruebaSemantica1():
5     x = x + 1
6     print ( x )
7
8 x = 9
9
10 print ('Usando el contenido de una variable')
11 pruebaSemantica()
12
13 print ('Usando el contenido de una variable y guardando el resultado en la misma variable')
14 pruebaSemantica1()
15
16 print ('La variable x después de las llamadas ', x)

Usando el contenido de una variable
Usando el contenido de una variable y guardando el resultado en la misma variable
Traceback (most recent call last):
  File "pruebaSemantica.py", line 14, in <module>
    pruebaSemantica1()
  File "pruebaSemantica.py", line 5, in pruebaSemantica1
    x = x + 1
UnboundLocalError: local variable 'x' referenced before assignment

-----
(program exited with code: 1)

Presione una tecla para continuar . . .
```

La semántica en Python es muy distinta:

- es fuertemente tipado (no puedo violar tipos de datos)
- no requiere declarar el tipo de una variable
- Cuando una variable recibe una asignación dentro de una función, pasa a ser calificada como **local** y tiene una visibilidad limitada al cuerpo de la función, deja de ser considerada como global si ya existía en el ámbito exterior
- Solución **definirla global** en el cuerpo de la función para que tome el valor global

SEMÁNTICA - EJEMPLOS EN C CON 3 ERRORES DISTINTOS

1

```
main.c
1  #include <stdio.h>
2
3  int main()
4  {
5      int a, resul;
6      char cadena;
7
8      cadena = 'h';
9      resul = a + x;
10     printf(resul);
11     return 0;
12 }
13
```

2

```
main.c
1  #include <stdio.h>
2
3  int main()
4  {
5      int a, resul;
6      char cadena;
7
8      cadena = 'h';
9      resul = a + cadena;
10     printf(resul);
11     return 0;
12 }
```

3

```
main.c
1  #include <stdio.h>
2
3  int main()
4  {
5      int a;
6      int a, b;
7
8      b = a * 2;
9      printf(b);
10     return 0;
11 }
```

1. La variable x no fué declarada.
2. En una expresión se combinan diferentes tipos de datos y no hay reglas que lo permitan o lo resuelvan.
3. Se declararon 2 variables con el mismo nombre en un mismo entorno.

SEMÁNTICA

- Hay que detectar otros errores semánticos
- Hay características de la estructura de los lenguajes de programación que son difíciles o imposibles de describir con las gramáticas *BNF/EBNF*.

Ejemplo: exigir que una variable esté declarada antes de ser utilizada implica recordar información previa del programa. Una gramática BNF puede reconocer una asignación válida, pero no puede verificar si la variable ya fue declarada, ya que no tiene memoria del contexto. por lo que estas verificaciones se delegan al análisis semántico.

- Se debe usar otro tipo de gramática
- Se debe ver que errores se pueden detectar antes de ejecutar y cuales durante ejecución

SEMÁNTICA

- **Tipos de semántica que analizaremos:**
 - Estática (antes de la ejecución)
 - Dinámica (durante la ejecución)

SEMÁNTICA ESTÁTICA

- Se la llama así porque el **análisis** para el **chequeo** se hace **sin necesidad de ejecutarlo**. (ej. Compilación)
- **No** está **relacionada** con el **significado** de la **ejecución** del **programa**, está más **relacionado** con las **formas válidas (con la sintaxis)**.
- El **análisis** está **ubicado** entre el **análisis sintáctico** y el **análisis de semántica dinámica**, pero más cercano a la **sintaxis**.

SEMÁNTICA ESTÁTICA

- ¿Cómo detectar errores de compatibilidad de Tipos? (ej. los vistos anterior C)
- ¿Cómo detectar errores de declaración de variables duplicadas? (ej. los vistos anterior C)
- ¿Como detectar errores de variables no declaradas antes de referenciarlas (ej. los vistos anterior Python)?

BNF/EBNF no nos sirve para estos casos.

Son gramáticas libres de contexto

no se basan en el significado si con las formas válidas

¿Son válidos estas sentencias?

"Mi perro canta una manta"

x = 10; int x;

¿Cómo lo verificamos?

SEMÁNTICA ESTÁTICA

Gramática de Atributos

- Las **gramáticas de atributos** son un **enfoque formal** inventado por Knuth en 1968. Se utilizan para **describir la sintaxis y la semántica estática**.
- Extienden a las **Gramáticas Libres al Contexto (BNF/EBNF)** y **agregan información adicional para dar significado**.
- Sirven tanto para **describir** como para **comprobar** la **corrección de las reglas semánticas estáticas de un programa**
- La **usan los compiladores antes de la ejecución**
- Generalmente **resuelven los aspectos de la semántica estática**.

SEMÁNTICA ESTÁTICA

Funcionamiento de la Gramática de atributos

- A las **construcciones** del lenguaje representadas por símbolos de la gramática (terminales o no terminales) se les asocian atributos que permiten describir y calcular información semántica del programa
- Un **atributo** puede ser:
 - valor de una **variable**,
 - **tipo** de una **variable** o **expresión**,
 - **ubicación** que ocupa una **variable** en memoria,
 - etc.
- Los **valores de los atributos** se obtienen mediante las llamadas “ecuaciones o reglas semánticas” asociadas a las **producciones gramaticales** (construcciones).

SEMÁNTICA ESTÁTICA

Funcionamiento de la Gramática de Atributos

- Las reglas gramaticales (producciones sintácticas) son similares a BNF.
- Las reglas semánticas (ecuaciones) permiten detectar errores y obtener el valor de los atributos.
- Los atributos están directamente relacionados a los símbolos gramaticales (terminales y no terminales)
- Las GA se suelen expresar en forma tabular para obtener el valor del atributo

◦ *Ejemplo: decl int X,Y*

Regla gramatical

Reglas semánticas (*obtener el valor*)

Regla 1



Ecuaciones de atributo asociadas

decl tipo lista-var

lista-var.at = tipo.at

Regla n

Ecuaciones de atributo asociadas

La tabla muestra, para cada regla gramatical, qué ecuaciones se usan para calcular atributos

SEMÁNTICA ESTÁTICA

Funcionamiento de la Gramática de Atributos

- Cada fila representa una **producción gramatical** **símil BNF/EBNF**
- La segunda columna indica las **reglas semánticas (ecuaciones)** asociadas a cada producción
- Cuando **en el árbol aparece esa producción** se **aplican automáticamente** esas **ecuaciones**
- Las **ecuaciones indican cómo calcular los atributos** a partir de otros atributos
- **No se “busca un valor”, sino que se calcula el valor del atributo**

SEMÁNTICA ESTÁTICA - EJEMPLOS

Gramática de atributos

- Ej.. Gramática simple para una “*declaración de variables*” sólo de *tipo int* y *float* en el lenguaje C. con atributo *at*

Reglas gramaticales

decl → *tipo lista-var*

tipo → int

tipo → float

lista-var → *id*

*lista-var*₁ → *id*, *lista-var*₂

Similar a BNF pero no igual!

<i>Cursiva</i>	No Terminales
Normal	Terminales
->	se define como
,	seguido
OR	no existe, hay que repetir fila

Reglas semánticas (atributos)

lista-var.at = *tipo.at*

tipo.at = int

tipo.at = float

id.at = *lista-var.at*

Añadetipo(id.entrada, lista-var.at)

id.at = *lista-var*₁.*at*

*Añadetipo(id.entrada, lista-var*₁.*at)*

*lista-var*₂.*at* = *lista-var*₁.*at*

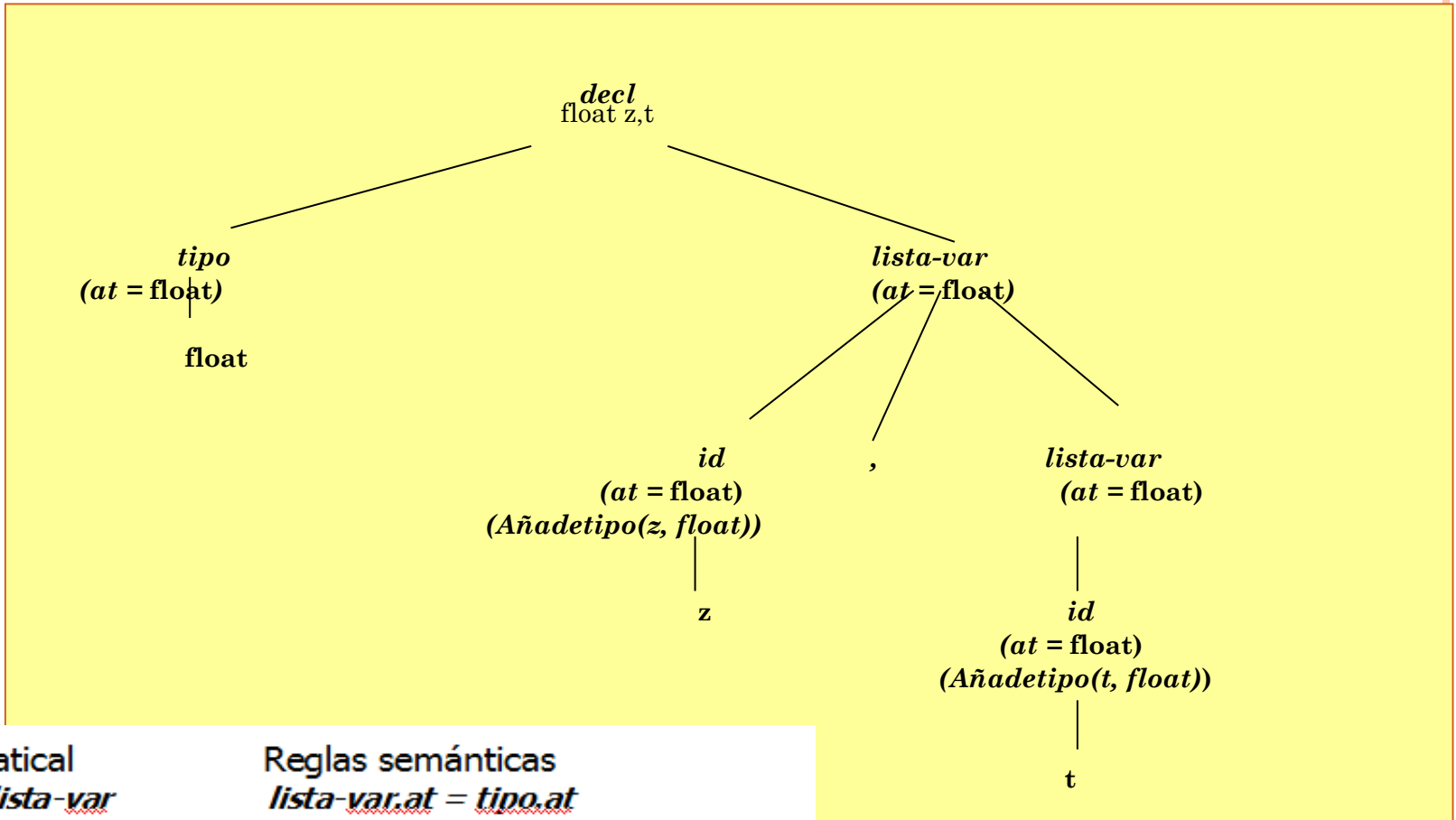
○ *id.entrada* referencia a la entrada del identificador en la tabla de símbolos

○ *Añadetipo* añade a Tabla de símbolos el tipo

○ Árbol sintáctico atribuido

Parte del Árbol sintáctico y de la gramática de atributos para ver si cumple y detectar errores

Ej.. muestra los cálculos de atributo para la declaración: “float z,t”



Regla gramatical
 $decl \rightarrow tipo\ lista-var$

$tipo \rightarrow int$

$tipo \rightarrow float$

$lista-var \rightarrow id$

$lista-var_1 \rightarrow id, lista-var_2$

Reglas semánticas

$lista-var.at = tipo.at$

$tipo.at = int$

$tipo.at = float$

$id.at = lista-var.at$

$Añadirtipo(id.entrada, lista-var.at)$

$id.at = lista-var.at$

$Añadirtipo(id.entrada, lista-var_1.at)$

$lista-var_2.at = lista-var.at$

Esto permite la detección de errores cuando se agregan símbolos a la tabla de símbolos.

Si hay errores no pasa a ejecución.

SEMÁNTICA ESTÁTICA

Gramática de atributos

De la ejecución de las ecuaciones:

- ingresar **símbolos y atributos** a la **tabla de símbolos**,
- detectar y dar **mensajes de error** al ingresar a la **tabla de símbolos**:
- detectar **variables iguales**,
- controlar **tipo y variables de igual tipo**,
- controlar ciertas **combinaciones no permitidas** (reglas específicas del lenguaje)

Se debe tratar de representar todo lo que es necesario que salte antes de la ejecución y que no pudo ser detectado sintácticamente.

SEMÁNTICA DINÁMICA

- Es la que **describe** el significado de ejecutar las diferentes **construcciones** en el lenguaje de **programación**.
- Su **efecto (significado)** se ve durante la ejecución del **programa**.
- **Influirá** la interacción con el **usuario** y **errores** de la **programación**

SEMÁNTICA DINÁMICA

- Los programas **sólo se pueden ejecutar** si son **correctos** tanto en:
 - **sintaxis**
 - **semántica estática.**
- Luego viene la **semántica dinámica**

Analizar un ejemplo en C

int notas[10];

- **Un vector de 10 elementos consecutivos en memoria de tipo entero.**
- **¿Qué pasa si exceden los límites del mismo? ¿O posición 10?**
- **Pero si todo es correcto voy a obtener el valor del array de 10 elementos consecutivos en memoria en ejecución.**

SEMÁNTICA DINÁMICA

¿Cómo se describe la semántica dinámica de un lenguaje?

- **No es fácil escribirla** ¿como describir el proceso que sigue una computadora al ejecutar un programa en un lenguaje específico?
- **No existen herramientas estándar fáciles y claras** como en el caso de la sintaxis (diagramas sintácticos y BNF)
 - **Es complejo describir relación entre entrada y salida del programa**
 - **Es complejo describir cómo se ejecutará en cierta plataforma.**

SEMÁNTICA DINÁMICA

Enfoques-Soluciones más utilizadas:

- Formales - complejas:

1. Semántica **axiomática**
2. Semántica **denotacional**

La usan los diseñadores de compiladores, para especificar o razonar sobre el comportamiento de los programas.

- No formal:

3. Semántica **operacional**

Se usan en los manuales de lenguajes

- En todos se definen **Reglas para comprobar la ejecución, la exactitud de un lenguaje, comparar funcionalidades de distintos programas, especificar el significado.**
- Se pueden **usar combinados**
- **No todos sirven para todos los tipos de lenguajes de programación!**

Veremos una descripción general para conocerlos

SEMÁNTICA DINÁMICA

○ Semántica Axiomática

- **Considera al programa como una máquina de estados donde cada instrucción provoca un cambio de estado (condiciones de estado previo y posterior).**
- **Se parte de axioma (verdad) que sirve para verificar "estados y condiciones" a probar**
- **Los constructores del lenguaje se formalizan describiendo como su ejecución provoca un cambio de estado.**
- **Se desarrolló para probar la corrección de los programas si se cumplen condiciones**
- **La notación empleada es el cálculo de predicados, un lenguaje lógico que permite expresar propiedades matemáticas sobre variables y estados del programa,**

SEMÁNTICA DINÁMICA

Semántica Axiomática

- Una **condición** se describe con un **predicado** que describe los valores de las variables en ese estado
- Existe una **condición anterior** y una **condición posterior** a la ejecución del constructor.
- Cada **sentencia (S)** se **precede** y se **continúa** con una *expresión lógica* que *describe* las *restricciones* y *relaciones* entre los datos.
- **Precondición** (condiciones de estado previo P)
- **Postcondición** (condiciones de estado posterior Q)
- Se acompaña de funciones transformadoras de predicados y reglas

Ejemplos:

a $\perp$ b _____
r c

Precondición (P): {b distinto de cero}

Sentencia (S): expresión a/b

Postcondición (Q): {a=b*c+r y r<b}

(P): { x = 5 }

(S): x = x + 1

(Q): { x = 6 }

SEMÁNTICA DINÁMICA

○ Semántica Denotacional

- Se basa en **modelos matemáticos** y describe el significado de programas mediante funciones (generalmente recursivas)
- Es más preciso para obtener y verificar resultados/significado, pero es menos legible, menos intuitivo
- Se diferencia de la axiomática:
 - Axiomática: describe el significado a través de **PREDICADOS** (sobre estados del programa)
 - Denotacional: describe el significado a través de **FUNCIONES** (generalmente funciones recursivas) que definen su significado matemático

SEMÁNTICA DINÁMICA

○ **Semántica Denotacional**

- **Define** el significado de cada **construcción del lenguaje** como una **función matemática**, estableciendo una **correspondencia** entre **sintaxis** y su **significado**
- **Describe** la **dependencia funcional** entre el **resultado** de la **ejecución** y sus **datos de entrada**

A cada construcción sintáctica le asocia una función que define su significado.

Veamos un ejemplo.....

SEMÁNTICA DENOTACIONAL - EJEMPLO FUNCIÓN RECURSIVA

Producción-Definir el significado matemático de convertir binario a entero con función matemática **Binario → **Entero****

1. Gramática del número binario

$\langle \text{Nbin} \rangle ::= 0 \mid 1 \mid \langle \text{Nbin} \rangle 0 \mid \langle \text{Nbin} \rangle 1$

2. Función semántica recursiva que representa su significado

$\text{FNbin}(0) = 0$ $\text{FNbin}(\langle \text{Nbin} \rangle 0) = 2 * \text{FNbin}(\langle \text{Nbin} \rangle)$

$\text{FNbin}(1) = 1$ $\text{FNbin}(\langle \text{Nbin} \rangle 1) = 2 * \text{FNbin}(\langle \text{Nbin} \rangle) + 1$

Binario 110 → significa un 6

muestra cómo
asignar
significado
matemático a
una
estructura
sintáctica.

$\text{FNbin}(\langle \text{Nbin} \rangle \mathbf{0})$

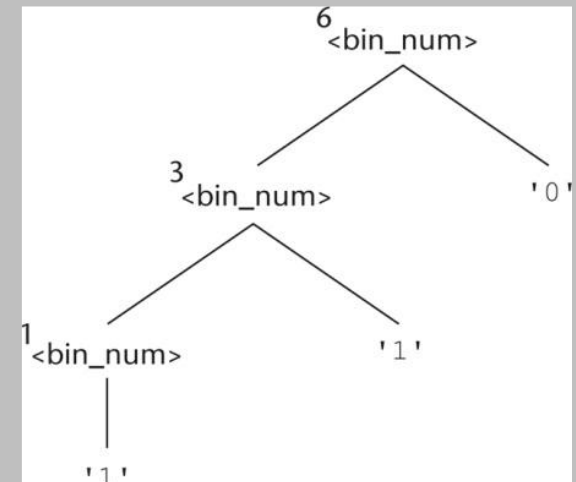
$2 * \text{FNbin}(\langle \text{Nbin} \rangle \mathbf{1})$

$2 * [2 * \text{FNbin}(\mathbf{1}) + 1]$

$2 * [2 * 1 + 1]$

$2 * [3]$

6 resultado de las funciones



SEMÁNTICA DINÁMICA

Semántica Operacional

- Define el significado de un programa describiendo cómo se ejecuta paso a paso.
- El comportamiento del programa se explica mediante operaciones de un lenguaje más simple o de bajo nivel, ejecutadas sobre una máquina abstracta.
- Durante la ejecución, cada instrucción produce un cambio en estado de la máquina.
- El estado representa los valores de las variables y la memoria en un momento determinado.
- Por lo tanto, el significado de un programa se describe como la secuencia de cambios de estado que se producen durante su ejecución.

SEMÁNTICA DINÁMICA

Semántica Operacional

- Es un **método informal** ya que se basa en **otro lenguaje de bajo nivel** y puede **llevar a cometer errores**
- Es uno de los **más utilizado** en los libros de texto para **explicar el significado de los lenguajes**
- **PL/1** fue el primero que lo utilizó en su definición

Utiliza otro lenguaje para describir el significado de los programas

SEMÁNTICA DINÁMICA

Semántica Operacional

Ejemplo: significado de una sentencia en PASCAL que se expresa mediante **operaciones de un lenguaje más simple**, ejecutadas sobre una **máquina abstracta**. (Ej. **Loop con goto**)

Lenguaje Pascal

```
for i := pri to ul do  
begin  
.....  
end
```

Máquina abstracta

```
i := pri (inicializo i)  
lazo if i > ul goto sal  
.....  
i := i + 1  
goto lazo  
sal .....
```

El significado del programa es = a la secuencia de cambios de estado durante su ejecución.

SEMÁNTICA DINÁMICA

Enfoques-Soluciones más utilizadas:

Enfoque de semántica	Idea principal	Cómo describe el programa	Uso principal
Semántica operacional	Describe la ejecución paso a paso	Mediante estados de una máquina abstracta y transiciones	Especificar cómo se ejecuta un programa
Semántica denotacional	Describe el significado matemático	Asocia cada construcción del lenguaje con una función matemática	Modelos formales de lenguajes
Semántica axiomática	Describe propiedades lógicas del programa	Usa axiomas y reglas de inferencia	Verificación de programas

SEMÁNTICA DINÁMICA

Enfoques-Soluciones más utilizadas:

Enfoque de semántica	Quién lo define o decide	Cuándo se usa	Quién lo ejecuta o aplica	Herramientas o implementaciones	Referencias bibliográficas
Semántica operacional	Diseñadores del lenguaje y diseñadores de compiladores	Durante el diseño del lenguaje y en la implementación del compilador o intérprete, manuales	intérpretes, compiladores o máquinas virtuales	Java Virtual Machine , intérprete de Python , intérpretes de lenguajes educativos	Concepts of Programming Languages; Semantics with Applications: An Appetizer
Semántica denotacional	Investigadores o diseñadores formales de lenguajes	Durante la especificación formal o análisis teórico de un lenguaje	normalmente no se ejecuta directamente; se usa como modelo matemático	marcos semánticos como K Framework , diseño de lenguajes funcionales como Haskell	Semantics with Applications: An Appetizer ; trabajos de Dana Scott
Semántica axiomática	Investigadores en verificación formal o ingenieros de software crítico	Cuando se quiere demostrar propiedades o corrección de programas	herramientas de verificación formal o asistentes de prueba	Coq, Isabelle , verificación en SPARK	trabajos de C. A. R. Hoare ; Semantics with Applications: An Appetizer

SEMÁNTICA DINÁMICA

Ejemplo donde se utilizan los distintos enfoques

INVESTIGACIÓN EN LENGUAJES

(comunidad académica)

- desarrollo de teorías semánticas
 - semántica operacional
 - semántica denotacional
 - semántica axiomática

DISEÑO DEL LENGUAJE

(diseñadores del lenguaje)

- especificación formal del lenguaje
 - operacional
 - denotacional

DOCUMENTACIÓN Y ESTÁNDARES

(manuales y especificaciones)

- descripción del comportamiento del lenguaje
 - usualmente estilo operacional

IMPLEMENTACIÓN DEL LENGUAJE

(compiladores / intérpretes)

- implementación concreta de la semántica
 - normalmente operacional

USO DEL LENGUAJE

(programadores)

- ejecución de programas basada en la semántica implementada

VERIFICACIÓN FORMAL

(ingeniería de software crítica)

- semántica axiomática para demostrar propiedades del programa

INVESTIGACIÓN Y HERRAMIENTAS

- asistentes de prueba
- frameworks semánticos
- análisis formal de programas

PROCESAMIENTO DE LOS PROGRAMAS

PROCESAMIENTO DE UN LENGUAJE

TRADUCCIÓN

- Las computadoras ejecutan un **lenguaje de bajo nivel** llamado “**lenguaje de máquina**” (con 0's y 1's)
- Hoy los **programas** están escritos en **lenguajes de más alto nivel**
- Un poco de historia...
 - Se Programaba en código de máquina (0's y 1's)
 - Esto era muy complejo
 - Tenía muchos errores
 - **Se pensaron soluciones**

PROCESAMIENTO DE UN LENGUAJE

TRADUCCIÓN

- Una **solución** fue reemplazar **repeticiones/ patrones de bits** por un **código nemotécnico** (una abreviatura del propósito de la instrucción).
 - Así surgen:
 - “**Lenguaje Ensamblador**” que usaba estos códigos para programar
- ```
SUM #10, #11, #13
SUM #13, #12, #13
DIV #13, 3, #13
FIN
```
- “**Programa Ensamblador**” que lo convertía a lenguaje de máquina

# PROCESAMIENTO DE UN LENGUAJE

## TRADUCCIÓN - EJEMPLO HOLA MUNDO

### Procesadores 80x86 de Intel

```
.data
msg:
.string "Hello, World!\n"
len:
.long . - msg
.text
.globl _start
_start:
push $len
push $msg
push $1
movl $0x4, %eax
call _syscall
addl $12, %esp
push $0
movl $0x1, %eax
call _syscall
_syscall:
int $0x80
ret
```

### Procesadores de la familia Motorola 68000

```
start:
move.l #msg, -(a7)
move.w #9, -(a7)
trap #1
addq.l #6, a7
move.w #1, -(a7)
trap #1
addq.l #2, a7
clr -(a7)
trap #1
msg: dc.b "Hello,
World!", 10, 13, 0
```

- Cada arquitectura o familia de procesadores tiene su propio SET de instrucciones.
- Cada uno su propio:
  - **Lenguaje ensamblador**
  - **Programa ensamblador**
  - **Código nemotécnico**Para llegar al código de máquina

# PROCESAMIENTO DE UN LENGUAJE

## TRADUCCIÓN - EJEMPLO HOLA MUNDO

- **Problemas** que se producían:
  - **imposible intercambiar** programas entre **distintas máquinas** o de distintas **familias de procesadores**
  - **diferentes versiones** para una **misma CPU** pueden tener **juegos de instrucciones incompatibles**
  - **modelos evolucionados** de una **familia de CPU** pueden incorporar **instrucciones nuevas**
- Se buscaron otras **soluciones**

# PROCESAMIENTO DE UN LENGUAJE

## TRADUCCIÓN

- **“Lenguajes de alto nivel”** que permitieron esta abstracción

**Ejemplo de lenguaje máquina para el microprocesador 68000:  
suma de dos enteros:**

| Dirección | Código Binario   | Código Ensamblador | Alto Nivel |
|-----------|------------------|--------------------|------------|
| \$1000    | 0011101000111000 | MOVE.W \$1200,D5   | Z=X+Y      |
| \$1002    | 0001001000000000 |                    |            |
| \$1004    | 1101101001111000 | ADD.W \$1202,D5    |            |
| \$1006    | 0001001000000010 |                    |            |
| \$1008    | 0011000111000101 | MOVE.W \$D5,\$1204 |            |
| \$100A    | 0001001000000100 |                    |            |



# PROCESAMIENTO DE UN LENGUAJE

## TRADUCCIÓN: INTERPRETACIÓN Y COMPILACIÓN

¿Cómo los **programas escritos** en un **lenguaje de alto nivel** pueden ser **ejecutados sobre una computadora** cuyo **lenguaje** es muy diferente y de muy **bajo nivel** que entiende **0s y 1s**?

- Con **Programas Procesadores del lenguaje**

### Alternativas:

- Interpretación
- Compilación
- Interpretación y Compilación (combinación)

*¡No es decisión del Programador!*

*Programador sólo elige el lenguaje que más le convenga.*

*¡Es decisión del Desarrollador del lenguaje!*



# INTERPRETACIÓN

- Hay un **Programa** escrito en **lenguaje de programación interpretado**
- Hay un **Programa** llamado **Intérprete** que realiza el **procesamiento** de ese lenguaje interpretado en el momento de **ejecución**

Ej.: Lisp, Smalltalk, Basic,  
Python, Ruby, PHP, Perl, etc.

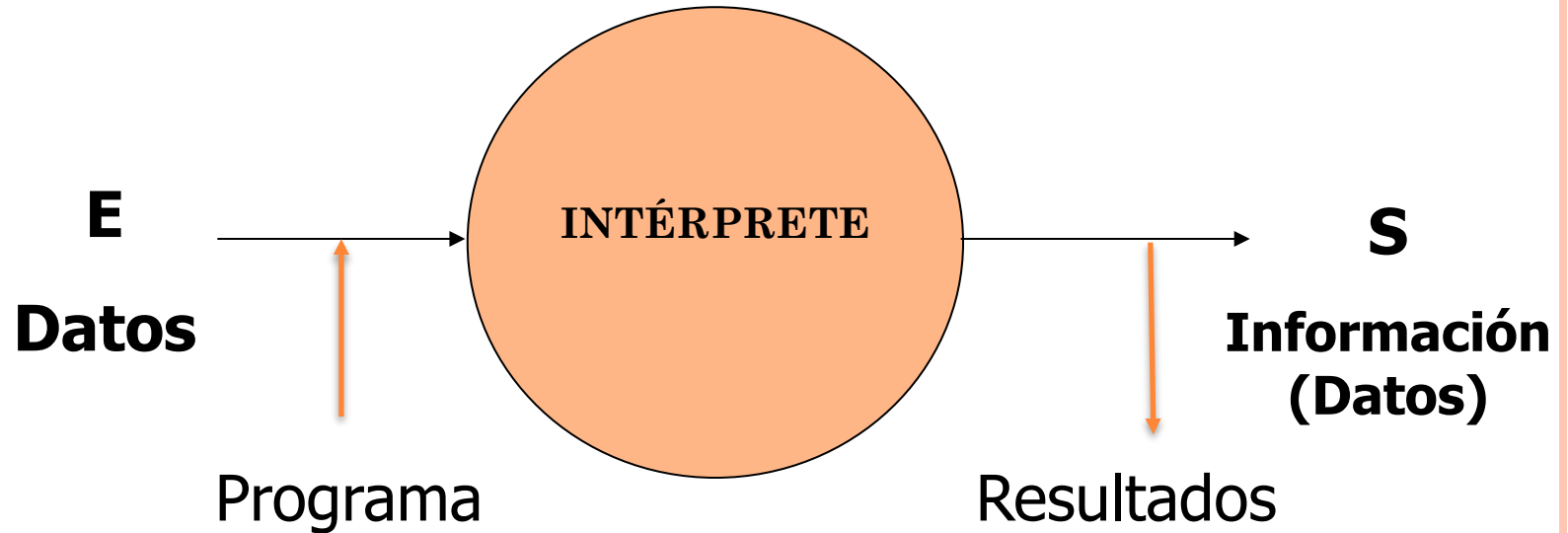
# INTERPRETACIÓN

- El **proceso** que realiza cuando se ejecuta **sobre cada una de las sentencias** del programa es:
  - Leer (próxima sentencia)
  - Analizar
  - Decodificar
  - Ejecutar (una a una las sentencias de programa)
- Sólo **pasa por ciertas instrucciones no por todas**, según sea la ejecución (**ventajas y desventajas**)
- Cada vez que vuelvo a **ejecutar** el programa se **repite** toda la secuencia

# INTERPRETACIÓN

- El **Intérprete** suele contar con una serie de **herramientas** para realizar el ***procesamiento / traducción a lenguaje de máquina***
  - Por cada constructor del lenguaje hay un **subprograma** en lenguaje de máquina que ejecuta esa acción.
  - La interpretación se realiza llamando a estos **subprogramas** en la **secuencia adecuada** hasta **generar el resultado de la ejecución**.

# INTERPRETACIÓN



- **Obtiene** la próxima sentencia
- **Analiza y decodifica** la acción a ejecutar
- **Ejecuta** la acción

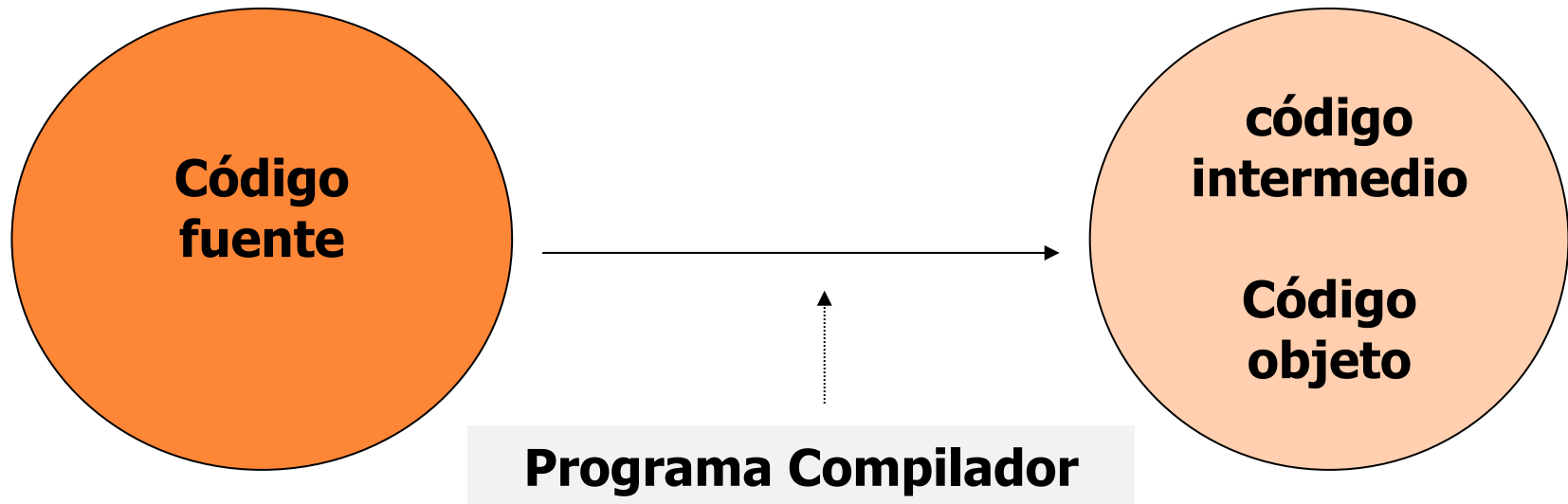
# COMPILACIÓN

- Hay un **programa** escrito en un **lenguaje de alto nivel**
- Hay un **programa** llamado **Compilador** que realiza la **traducción** a **lenguaje de máquina**
- Ej.: Ada, C, C ++, Fortran, Pascal, etc.)

# COMPILACIÓN

- El **compilador**, antes de la **ejecución**, toma **todo el programa** en un **lenguaje de alto nivel (lenguaje fuente)** y lo traduce a un **equivalente a lenguaje de máquina**.
- **Pasa por todas las instrucciones** antes de la ejecución (**ventajas y desventajas**)
- El **código** que se genera se **guarda** y se puede **reusar** ya **compilado**.
- La **compilación implica varias etapas**, que analizaremos luego.....

# COMPILACIÓN



Se puede generar:

- **Código objeto** generalmente el **ejecutable en lenguaje de máquina**
- **Código de nivel intermedio** **en lenguaje ensamblador**

# TRADUCCIÓN

## Comparación entre Compilador e Intérprete

- Por cómo se ejecuta:

- Intérprete:*

- Se utiliza el intérprete en la ejecución
    - Ejecuta el programa línea por línea
    - Por donde pase **dependerá** de la **acción** del **usuario**, de la **entrada de datos** y/o de alguna **decisión** del programa
    - Siempre se debe **tener el Programa Intérprete**
    - El programa fuente será **público** (necesito ambos)

- Compilador:*

- Se utiliza el compilador **antes de la ejecución**
    - Produce un **programa ejecutable** equivalente en **lenguaje objeto**
    - El programa fuente **no será publico**



# TRADUCCIÓN

## Comparación entre Compilador e Intérprete (cont..)

- Por el orden de ejecución:

- Intérprete:*

- Sigue el **orden lógico** de ejecución (**no necesariamente recorre todo el código**)

- Compilador:*

- Sigue el **orden físico** de las sentencias (**recorre todo**)

# TRADUCCIÓN

## Comparación entre Compilador e Intérprete (cont..)

### ○ Por el tiempo de ejecución del:

#### *Intérprete:*

- Por cada sentencia que pasa realiza el proceso de **decodificación (lee, analiza y ejecuta)** para determinar las operaciones y sus operandos. **Es repetitivo**
- Si la sentencia está en un **proceso iterativo** (ej.: for/while), se realizará la tarea de decodificación **tantas veces como sea requerido**
- La **velocidad** de proceso se puede ver **afectada** por esto

#### *Compilador:*

- Pasa por **todas las sentencias**.
- **No repite lazos**
- **Traduce todo de una sola vez.**
- Genera **código objeto ya depurado y compilado.**
- La **velocidad de compilar** dependerá del **tamaño del código**

# TRADUCCIÓN

## Comparación entre Compilador e Intérprete (cont..)

### ○ Por la eficiencia posterior:

#### Intérprete:

- Más lento en ejecución. Se repite el proceso cada vez que se ejecuta el mismo programa o pasa por las mismas instrucciones
- Para ser ejecutado en otra máquina se necesita tener si o si el intérprete instalado y el programa fuente será publico

#### Compilador:

- Más rápido ejecutar desde el punto de vista del hardware porque ya está en un lenguaje de más bajo nivel y listo para ser ejecutado.
- Ya compilado es más eficiente.
- De acuerdo al código tal vez se tardó más en compilar porque se verificó y detectó todo previamente.

# TRADUCCIÓN

## Comparación entre Compilador e Intérprete (cont..)

- **Por el espacio ocupado - *Intérprete*:**
- El programa no se traduce completamente antes de ejecutarse. Se interpreta instrucción por instrucción según la ejecución.
- La memoria se ocupa con:
  - El programa fuente, se mantiene en su forma original (texto de programa, árbol sintáctico)
  - La tabla de símbolos, variables y otras estructuras que se generen cuando se usan en forma dinámica
  - El intérprete contiene los subprogramas necesarios para ejecutar todas las instrucciones (*asignación, if, bucle, etc.*)
  - El intérprete. Puede cargarse una sola vez y luego ejecutar distintos programas (ej. **Python y Ruby**)
- **Importante:** El espacio ocupado depende del tamaño del programa y del intérprete. Si el programa es muy pequeño, el intérprete completo puede ocupar más memoria que un ejecutable compilado

# TRADUCCIÓN

## Comparación entre Compilador e Intérprete (cont..)

- **Por el espacio ocupado - Compilador**

- El programa se traduce completamente antes de ejecutarse, generando un **programa ejecutable**.
- • La memoria se utiliza para:
  - **El programa compilado (código máquina)**. Recordar que **varias instrucciones de máquina** pueden generarse a partir de una sola instrucción de alto nivel
  - **Datos del programa** durante la ejecución
  - **Bibliotecas necesarias para la ejecución**
  - **El compilador no necesita estar en memoria durante la ejecución, solo el programa ejecutable.**
- **Ventaja:** La ejecución suele ser **más rápida**, programa ya traducido.
- **Importante:** El espacio ocupado **depende del tamaño del programa y de la implementación del lenguaje**, por lo que un **programa compilado no siempre ocupa más memoria que uno interpretado**.

# TRADUCCIÓN

## Comparación entre Compilador e Intérprete (cont..)

- Por el espacio ocupado
- No es correcto afirmar simplemente:
  - “el intérprete ocupa menos memoria”*
  - “el compilado ocupa más memoria”*
- En realidad depende de dos factores:
  - El tamaño del programa que se desea ejecutar
  - La forma en que el lenguaje está implementado (compilador o intérprete)

Porque:

- en **interpretación** hay que cargar el intérprete completo
- en **compilación** si el programa es grande, el ejecutable también puede ser grande

# TRADUCCIÓN

## Comparación entre Compilador e Intérprete (cont..)

- **Por la detección de errores:**

- Intérprete:**

- Las **sentencias del código fuente** pueden ser **relacionadas directamente con la sentencia en ejecución**, entonces se puede ubicar donde se produjo el error.
    - Es más fácil detectarlos por donde pasa la ejecución
    - Es más fácil corregirlos

- Compilador:**

- Se pierde la **referencia** entre el **código fuente** y el **código objeto**.
    - Es **casi imposible ubicar el error**
    - Se deben **usar otras técnicas** (ej. **Semántica Dinámica**)

# TRADUCCIÓN - COMPARACIÓN AMPLIADA: INTERPRETACIÓN VS COMPILACIÓN

| Aspecto                   | Interpretación                               | Compilación                                  |
|---------------------------|----------------------------------------------|----------------------------------------------|
| Momento de traducción     | Durante la ejecución                         | Antes de ejecutar el programa                |
| Unidad de ejecución       | Sentencia por sentencia                      | Programa completo                            |
| Velocidad de ejecución    | Más lenta                                    | Más rápida                                   |
| Detección de errores      | Aparecen cuando se ejecuta la sentencia      | Muchos errores se detectan antes de ejecutar |
| Depuración                | Más fácil (ejecución paso a paso)            | Puede requerir recompilación                 |
| Uso de memoria            | Intérprete + programa + datos                | Ejecutable compilado + datos                 |
| Portabilidad              | Alta si existe intérprete para la plataforma | Requiere recompilar para cada arquitectura   |
| Distribución del programa | Se distribuye el código fuente o script      | Se distribuye un ejecutable                  |



# TRADUCCIÓN - TÉCNICAS UTILIZADAS

1. *Interpretación pura*
  2. *Compilación pura*
  3. *Traducción híbrida*
- **Combinación de Técnicas de Traducción:**
    - *Interpretación y Compilación*
    - *Compilación y Interpretación*

En ciertos **entornos de programación** se **combinan** para sacar **provecho** de sus **diferencias**, **reporte de errores**, etc.

# TRADUCCIÓN

- **Interpretación y Compilación**
  - El intérprete se utiliza en la **etapa de desarrollo** para facilitar el **diagnóstico** y **depuración de errores**.
  - Se **compila** el **programa validado** para generar un **código objeto más eficiente**.

**Ej. QuickBasic, Lisp, Scheme, etc.**

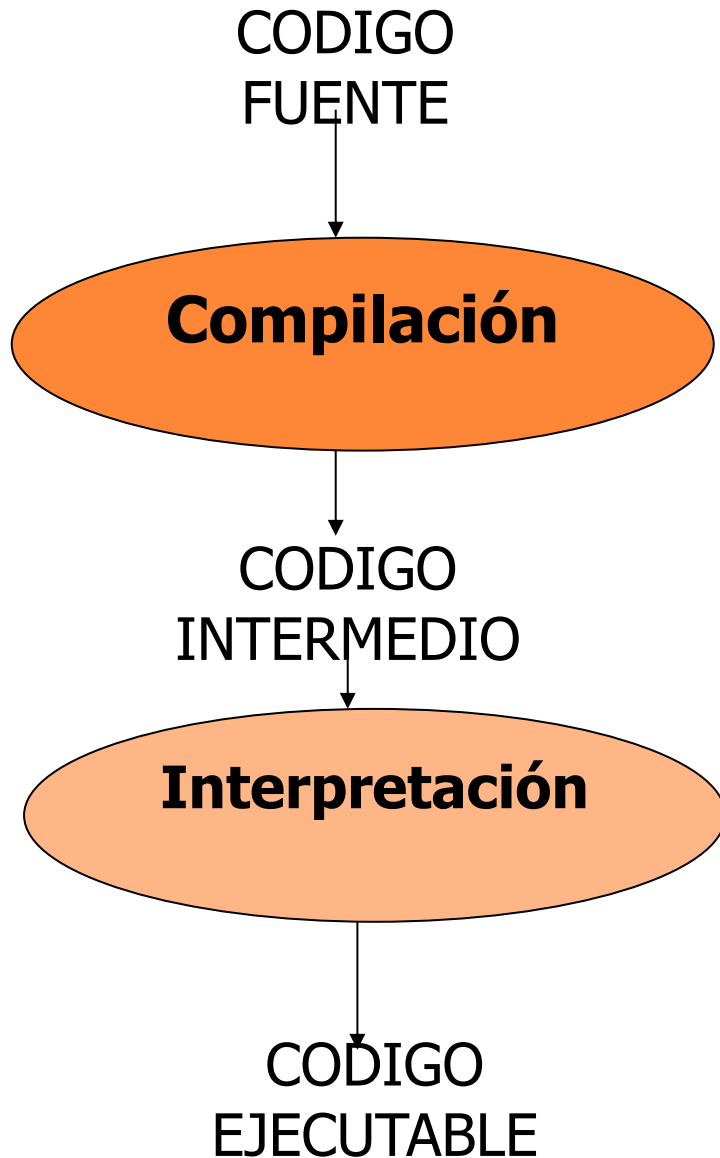
# TRADUCCIÓN

- **Compilación a código intermedio + ejecución en MV**
  - Se realiza **traducción inicial a un código intermedio independiente de la máquina.**  
Ese código luego se **ejecuta en una máquina virtual o sistema de ejecución.**
  - Esto permite generar **programas portables**, es decir, que pueden ejecutarse en diferentes máquinas y arquitecturas.

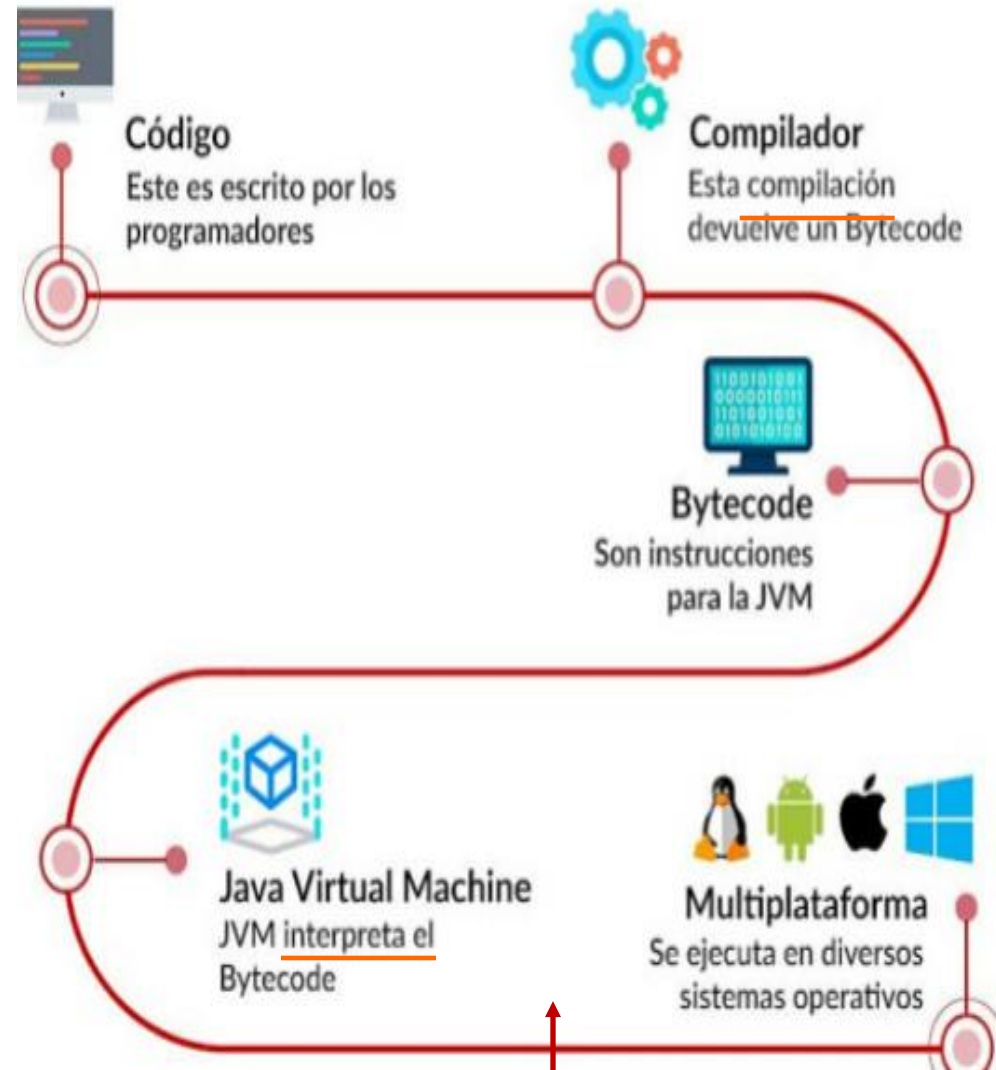
## Ejemplos:

- **Java (compilador Java genera bytecode. Este es ejecutado por Java Virtual Machine (JVM) en la máquina destino)**
- **C Sharp y VB.NET (Visual Basic)**
- **Python**

# TRADUCCIÓN: EJEMPLO COMBINACIÓN



## ¿Cómo funciona Java?



**JIT compila a código de máquina, elimina código que se repite para agilizar**

# COMPILADORES

## FUNCIONAMIENTO

# COMPILADORES – CÓMO FUNCIONAN

- **Traducen todo el programa**
- Pueden **generar un "código ejecutable" (.exe)**
- Pueden **generar un "código intermedio" (.obj)**
- La **compilación** puede ser en 1 o 2 etapas

**Ej.: C, Ada, Pascal, etc.**

# COMPILADORES – CÓMO FUNCIONAN

## ○ Estructura de un compilador. Etapas:

### 1. Etapas de Análisis

- Análisis léxico (Scanner)
- Análisis sintáctico (Parser)
- Análisis semántico (Semántica estática)

Trabaja sobre el programa fuente. Depende del lenguaje

### 2. Etapas de Síntesis

- Optimización del código
- Generación del código final

Puede generarse:  
código intermedio y  
optimización

Genera el programa objeto, Depende de la máquina

# COMPILADORES

## 1) Etapa de Análisis del programa fuente

### ◦ Análisis léxico (Scanner):

- Es un proceso extenso
- Reconoce lexemas (cadena de caracteres) para luego reconocer Tokens (categorías léxicas)
- Utiliza reglas léxicas para reconocer los Tokens válidos
- Se identifican/clasifican: identificadores, números, operadores, delimitadores, símbolos especiales, palabras clave, palabras reservadas, comentarios, etc.
- Se filtran comentarios y separadores (como: espacios en blanco, tabulaciones, etc.)



# COMPILADORES

## 1) Etapa de Análisis del programa fuente

### ◦ Análisis léxico (Scanner) cont.:

- Mantiene o consulta la **tabla de símbolos**.
- Inserta los **identificadores** nuevos en la **tabla de símbolos**. El token generado **incluye una referencia a esa entrada de la tabla de símbolos**.
- **Genera errores** si la secuencia de caracteres **no coincide con ninguna categoría léxica**.
- El resultado de esta etapa es una **secuencia de tokens válidos** y la **detección de errores léxicos**.

.

# COMPILADORES

- Ejemplo de token (componente léxico), lexema y regla

| ID  | Componente léxico<br>token | Lexema                 | Patrón                          |
|-----|----------------------------|------------------------|---------------------------------|
| 201 | IF                         | if                     | if                              |
| 202 | OP_RELACIONAL              | <, >, !=, ==           | <, >, !=, ==                    |
| 203 | IDENTIFICADOR              | var, s1,<br>suma, prom | letra (letra  <br>dígito   gb)* |
| 204 | ENTERO                     | 2, 23, 5124            | (dígito)+                       |

# COMPILADORES

- Ejemplo de salida de un analizador léxico

```
x:=a+b*c;
y:=3+b*c;
```



**Categorías** →

| TOKENS RECONOCIDOS |             |            |
|--------------------|-------------|------------|
| (id, "x")          | (op, ":=")  | (id, "a")  |
| (op, "+")          | (id, "b")   | (op, "*")  |
| (id, "c")          | (punt, ";") |            |
| (id, "y")          | (op, ":=")  | (num, "3") |
| (op, "+")          | (id, "b")   | (op, "*")  |
| (id, "c")          | (punt, ";") |            |

# COMPILADORES

- Ejemplo de salida de un analizador léxico y tabla de símbolos

Código fuente

**x := a + b \* 10;**



Analizador Léxico



**x    :=    a    +    b    \*    10    ;**

Secuencia de Tokens

**(id,1) (assign, :=) (id,2) (op,+) (id,3) (op,\*) (num,10) (punt,;)**



Con ID a tabla de símbolos

Tabla de Símbolos

ID   Lexema   Otros.....

**1   x**

**2   a**

**3   b**

# COMPILADORES

## 1) Etapa de Análisis del programa fuente

### ○ Análisis sintáctico (Parser)

- **Recibe los tokens generados por el Analizador Léxico.**
- **Analiza la estructura del programa (*sentencias, expresiones, declaraciones*).**
- **Verifica si la secuencia de tokens cumple la gramática del lenguaje.**
- **Detecta y reporta errores sintácticos.**

# COMPILADORES

## 1) Etapa de Análisis del programa fuente

### ○ Análisis sintáctico (Parser)

- Utiliza las gramáticas formales que describen la sintaxis del lenguaje.
- Intenta construir un árbol de análisis sintáctico (o el árbol de derivación) según gramática
- Si no puede construir el árbol se detecta error sintáctico

### ○ El árbol permite:

- representar la estructura jerárquica del programa
- mostrar cómo la sentencia se deriva según la gramática del lenguaje
- servir de entrada para el análisis semántico

# COMPILADORES

## ○ Ejemplo Fase léxica y Fase Sintáctica

### Programa Fuente

```
x:=a+b*c;
y:=3+b*c;
```

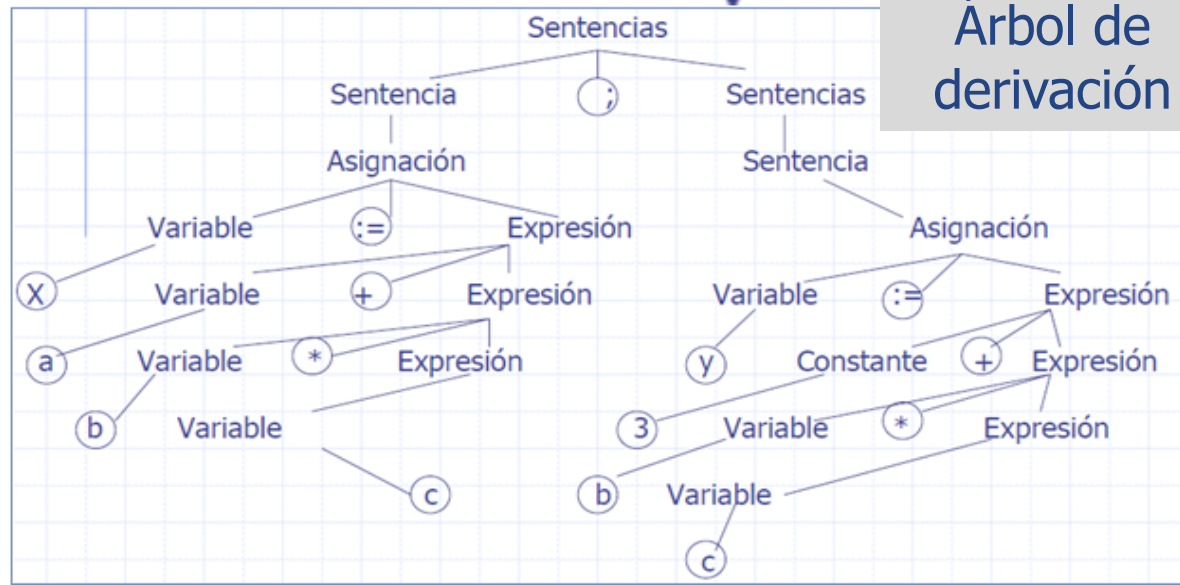
Analizador  
léxico

| TOKENS   |            |           |
|----------|------------|-----------|
| (id,"x") | (op,":=")  | (id,"a")  |
| (op,"+") | (id,"b")   | (op,"*")  |
| (id,"c") | (punt,";") |           |
| (id,"y") | (op,":=")  | (num,"3") |
| (op,"+") | (id,"b")   | (op,"*")  |
| (id,"c") | (punt,";") |           |

Analizador  
Sintáctico

Reglas

Árbol de  
derivación



# COMPILADORES

## 1) Etapa de Análisis del programa fuente

### ○ Análisis semántica (semántica estática):

- Fase clave del análisis del programa
- Se ejecuta después del análisis léxico (Scanner) y sintáctico (Parser).
- Trabaja sobre las estructuras sintácticas ya reconocidas por el Parser.
- Verifica que el programa tenga significado válido según las reglas del lenguaje.
- Agrega información semántica al programa (*tipos, alcance, referencias*).
- El programa continúa tomando forma para la generación de código.

**El análisis semántico agrega significado a la estructura del programa mediante atributos**



# COMPILADORES

## 1) Etapa de Análisis del programa fuente

### ○ Análisis semántica (semántica estática):

- **Comprobación y control de tipos** (*compatibilidad entre variables, expresiones y operaciones*).
- **Gestión de la tabla de símbolos.** Se agregan o completan atributos como: *tipo (int, float), alcance (local, global), clase de identificador (variable, función, constante, etc.)*
- **Verificación de nombres:** las variables declaradas o no antes de usarse, control de duplicados en un mismo ámbito
- **Aplicación de reglas semánticas del lenguaje** (mediante gramáticas de atributos, árbol atribuido)
- **Es puente entre la Etapa de Análisis y la de Síntesis**

# COMPILADORES

## 2) Etapa de Síntesis del compilador

### ○ Generación de Código Intermedio

- Consiste en traducir el programa fuente a una representación intermedia pensada para una máquina abstracta.
- Es independiente de la arquitectura del hardware.
- Facilita la optimización y la posterior generación de código objeto.

**El código objeto final sí depende de la máquina (procesador, arquitectura).**

# COMPILADORES

## 2) Etapa de Análisis del programa fuente

### ○ Generación a código intermedio:

- El código intermedio debe cumplir:

- ser fácil de generar a partir del árbol sintáctico
- ser fácil de traducir a código objeto

- Técnica utilizada:

**Código de tres direcciones:** Se representa el programa como una secuencia de instrucciones simples, por ejemplo:

**$x := y \text{ op } z$**

**donde:  $x$  = variable destino,  $y, z$  = operandos,  $op$  = operador**

Muchas expresiones complejas se **descomponen en pasos simples** usando **variables temporales**

# COMPILADORES

## EJEMPLO DE GENERACIÓN DE CÓDIGO INTERMEDIO:

### Código de 3-direcciones:

**$A := B \text{ op } C$**

- **$A, B, C$**  son operandos/variables
- **$op$**  es un operador binario
- Se permiten condicionales simples (IF)
- Se permiten saltos. (GOTO)
- Cada sentencia se traduce en N líneas

```
while (a > 0) and (b < (a*4 - 5))
 a := b*a - 10;
```

```
L1: if (a > 0) goto L2
 goto L3
```

```
L2: t1 := a * 4
 t2 := t1 - 5
 if (b < t2) goto L4
 goto L3
```

```
L4: t1 := b * a
 t2 := t1 - 10
 a := t2
 goto L1
```

```
L3: ...
```

# COMPILADORES

## 2) Etapa de Análisis del programa fuente

### ○ Optimización de código intermedio:

#### Eliminación de expresiones constantes (constant folding)

$x := 3 * 4$

se reemplaza por:  $x := 12$

---

#### Eliminación de código muerto

$t1 := a * b$

$t1$  nunca se usa, se elimina.

---

#### Eliminación de subexpresiones comunes

$t1 := a + b$

se optimiza a:

$t2 := a + b$

$t1 := a + b$

$t2 := t1$

---

#### Propagación de copias

$t1 := a$

se optimiza a:

$b := t1$

$b := a$

---

#### Simplificación algebraica

$x := y * 1$

se optimiza a:

$x := y$

# COMPILADORES

## • 2) Etapa de Síntesis:

- Se construye el programa ejecutable en esta etapa
- Se traduce el código intermedio a instrucciones máquina.
- Optimización de código (opcional), mejorar el programa para que sea más **eficiente** (más **rápido** o más **pequeño**).
- **Enlace de módulos (Linkeditor)**. Si el programa está dividido en varios módulos o usa bibliotecas, el **linker combina todos los módulos objeto en un único programa ejecutable**.  
(*módulos del programa, funciones, librerías, subrutinas, procedimientos*)
- El resultado final es un **programa ejecutable o módulo de carga**.
- Interviene el **Loader (cargador)** del sistema operativo, que **carga el programa en memoria para su ejecución**.

# COMPILADORES

- **2) Etapa de Síntesis:**

- **Optimización del código:**

- La optimización **no siempre se realiza y depende del compilador y de las opciones de compilación.**

- **Es una fase opcional.**
- **Puede estar integrada dentro del compilador.**
- **Puede existir como herramienta independiente.**
- En muchos compiladores la optimización se activa mediante **opciones de compilación.**

**Ejemplo: gcc -O2 programa.c**

**Las distintas opciones (flags) indican el nivel de optimización que se desea aplicar.**

# COMPILADORES

## ○ 2) Etapa de Síntesis:

### ○ Tipos de Optimización del código GCC

- **Principales Niveles de Optimización (-O):**
  - **-O0: Predeterminado.** Sin optimización, compilación rápida, ideal para depuración.
  - **-O1: Optimización básica,** reduce el tiempo de compilación.
  - **-O2: Optimización estándar. Recomendado** para la mayoría de las aplicaciones. Aumenta el **rendimiento sin incrementar** significativamente el **tamaño del código**.
  - **-O3: Optimización alta.** Habilita **optimizaciones agresivas** como *inlining* de funciones y vectorización.
  - **-Os: Optimiza para tamaño.** Habilita -O2 pero deshabilita optimizaciones que aumentan el tamaño del binario.
  - **-Ofast: Ignora estándares** estrictos (como IEEE math). Activa -O3 y -ffast-math.
  - **-Og: Optimización para depuración.** Selecciona optimizaciones que no interfieren con el *debugging*.



# COMPILADORES

## ○ 2) Etapa de Síntesis:

### ○ Optimización del código:

Un compilador puede aplicar diversas optimizaciones, por ejemplo:

- Reducir el tiempo de ejecución del programa
- Reducir el tamaño del código ejecutable

Algunas optimizaciones comunes son:

- Eliminación de código muerto (código que nunca se ejecuta)
- Eliminación de funciones no utilizadas
- Simplificación de expresiones algebraicas
- Evaluación de expresiones constantes en compilación
- Mejora de la evaluación de expresiones booleanas
- Generación de código optimizado para un procesador específico
- Comprobación de rangos
- Detección de desbordamiento de pila

# COMPILADORES

## ○ Ejemplo de optimización: eliminación de saltos innecesarios

Cuando hay **dos saltos consecutivos**, el compilador puede **simplificar la condición y eliminar uno de los saltos**

### Código inicial

```
L1: if (a>0) goto L2
 goto L3
L2: t1:=a*4
 t2:=t1-5
 if (b < t2) goto L4
 goto L3
L4: t1:=b*a
 t2:=t1-10
 a:=t2
 goto L1
L3:
```

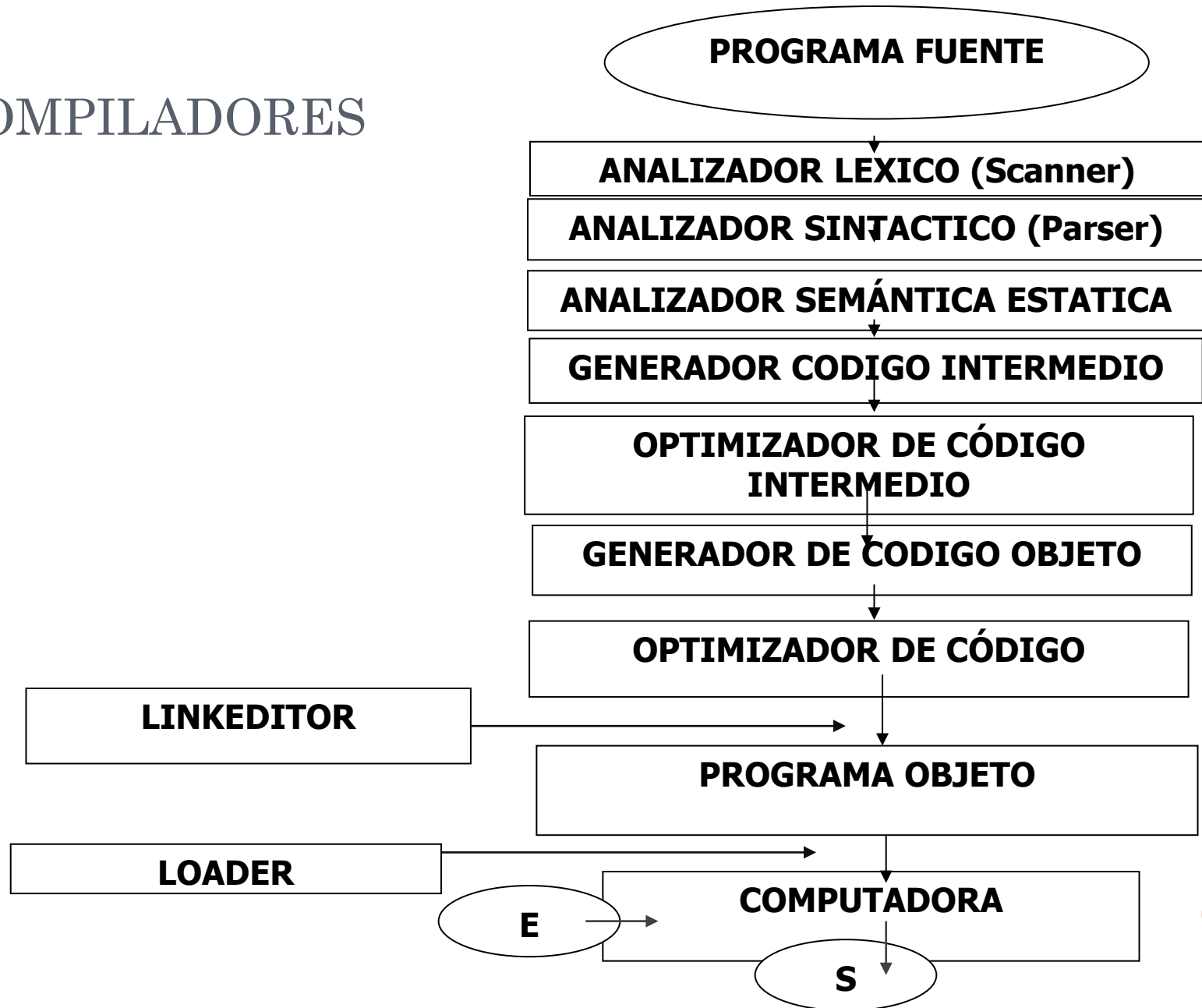
### Código optimizado

```
L1: if (a<=0) goto L3
 t1:=a*4
 t2:=t1-5
 if (b >= t2) goto L3
 t1:=b*a
 t2:=t1-10
 a:=t2

L3: goto L1
```

Se **elimina** instrucción **goto**, que produce un **código más simple y eficiente**. Optimización aplicada: **simplificación del flujo de control**.

# COMPILADORES



# PRÓXIMA CLASE

## SEMÁNTICA OPERACIONAL.

- Ligadura. Descriptores. Momentos de ligadura. Estabilidad.
- Variables. Arquitectura Von Newman. Atributos. Momentos y estabilidad. Nombre: características. Alcance: visibilidad, reglas. Tipo: definición, clasificación. L-valor: tiempo de vida, alocação. R-valor: constantes, inicialización. Alias